

Rules for variable names ¶

```
In [1]: ### alpha-numeric and underscores only
name_of_ama = "Ama" # all alphabets
name_of_ama
```

```
age1 = 25 # kofi's age
age1
```

```
kofi_age = 22
kofi_age
```

```
a1ge_23 = 23
```

```
_job = 45
_job
```

```
kb.asnte = 123 ## expect error because you cannot use . in variable names
```

```
-----
NameError                                Traceback (most recent call last)
```

```
Cell In[1], line 16
```

```
13 _job = 45
```

```
14 _job
```

```
----> 16 kb.asnte = 123
```

```
NameError: name 'kb' is not defined
```

```
In [2]: ### case sensitive
name # this is what has been defined
```

```
Name # this give error because it start with N but what has been
#defined start with n
```

```
-----
NameError                                Traceback (most recent call last)
```

```
Cell In[2], line 2
```

```
1 ### case sensitive
```

```
----> 2 name # this is what has been defined
```

```
4 Name
```

```
NameError: name 'name' is not defined
```

```
In [3]: ##### motivattion for functions
from math import sin
def ratio(a,b):
    sin_a = sin(a)
    sin_b = sin(b)

    result = sin_a/sin_b
    return(result)

ratio(2,3)
```

Out[3]: 6.443433778999866

```
In [1]: sum([2,4])
```

Out[1]: 6

```
In [5]: ### no python keyword
sum = 2+1
sum
```

Out[5]: 3

```
In [8]: sum([2,4])
```

...

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

Data types

```
In [3]: myint = 15
myint
```

Out[3]: 15

```
In [4]: myfloat = 1.0
myfloat
```

Out[4]: 1.0

```
In [5]: mycomplex = 2+3j  
mycomplex
```

Out[5]: (2+3j)

```
In [6]: mycomplex2 = complex(2,3)  
mycomplex2
```

Out[6]: (2+3j)

```
In [12]: mybool1 = True; mybool2 = False  
print(mybool1,mybool2)
```

True False

```
In [13]: mystr = 'My God is good'  
mystr
```

Out[13]: 'My God is good'

```
In [15]: mylist = ['Dad', 12, 1.56, 45+2j]  
mylist
```

Out[15]: ['Dad', 12, 1.56, (45+2j)]

```
In [16]: mytuple = ("dan", 34, 3.6+5j)  
mytuple
```

Out[16]: ('dan', 34, (3.6+5j))

```
In [17]: myset = {"Moses", 56}  
myset
```

Out[17]: {56, 'Moses'}

```
In [18]: myset = {12, "fortunately"}  
myset
```

Out[18]: {12, 'fortunately'}

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

Arithmetic operators

```
In [23]: ### Addition  
add = 1+3  
print(add)
```

4

```
In [24]: ### subtraction  
sub = 1-3  
print(sub)
```

-2

```
In [25]: ### multiplication  
multiply = 2*4  
multiply
```

Out[25]: 8

```
In [27]: ### division  
quotient = 2/6  
print(quotient)
```

0.3333333333333333

```
In [29]: ### modulus  
mod = 5%2  
print(mod)
```

1

```
In [30]: ### floor division  
myfloor = 5//2  
print(mylfloor)
```

2

```
In [33]: print("The fraction 5/2 gives "+str(5//2)+" whole number, and "+str(5%2)+str(")
```



The fraction 5/2 gives 2 whole number, and 1/2

```
In [34]: ### exponent  
myexp = 2**5  
print(myexp)
```

32

In []:

In []:

In []:

Assignment Operators

In [47]: *### add-assign*

```
a = 5
a += 5
a
```

Out[47]: 10

In [48]: *### subtract-assign*

```
a = 3
a -= 3
a
```

Out[48]: 0

In [49]: *### multiply-assign*

```
a = 6
a *= 5
a
```

Out[49]: 30

In [51]: *### divide-assign*

```
a = 7
a /= 3
a
```

Out[51]: 2.3333333333333335

In [52]: *### floor divide-assign*

```
a = 8
a //= 5
a
```

Out[52]: 1

In [53]: *### modulus-assign*

```
a = 5
a %= 3
a
```

Out[53]: 2

```
In [54]: ### exponent-assign  
a = 6  
a **= 3  
a
```

Out[54]: 216

```
In [55]: from math import cos,sin
```

```
In [59]: 1 == sin(0)+cos(0)
```

Out[59]: True

```
In [61]: 1 < sin(0)+cos(0)
```

Out[61]: False

```
In [63]: 1 <= sin(0)+cos(0)
```

Out[63]: True

```
In [65]: 1 > sin(0)+cos(0)
```

Out[65]: False

```
In [66]: 1 >= sin(0)+cos(0)
```

Out[66]: True

```
In [67]: 1 != sin(0)+cos(0)
```

Out[67]: False

```
In [69]: list = [1,2,3,4,5]  
3 in list
```

Out[69]: True

```
In [70]: list = [1,2,3,4,5]  
7 not in list
```

Out[70]: True

```
In [71]: a = 2
```

In [72]: `25/2-1+6*7`

Out[72]: 53.5

In [73]: `25/2+1-6*7`

Out[73]: -28.5

In []:

In []:

Complex numbers

In [74]: `mycomp = complex(2,3)`
`mycomp`

Out[74]: (2+3j)

In [75]: `mycomp.real`

Out[75]: 2.0

In [76]: `mycomp.imag`

Out[76]: 3.0

In [79]: `mycomp.conjugate()`

Out[79]: (2-3j)

In [80]: `abs(mycomp)`

Out[80]: 3.605551275463989

In [81]: `mydict = {"real": mycomp.real, "imag": mycomp.imag, "conjugate": mycomp.conjugate()}`
`mydict`

Out[81]: {'real': 2.0, 'imag': 3.0, 'conjugate': (2-3j), 'magnitude': 3.605551275463989}

In [82]: `mycomp = complex(5,7)`
`mycomp`

Out[82]: (5+7j)

```
In [83]: mydict = {"real":mycomp.real, "imag":mycomp.imag, "conjugate":mycomp.conjugate}
mydict
```

```
Out[83]: {'real': 5.0, 'imag': 7.0, 'conjugate': (5-7j), 'magnitude': 8.602325267042627}
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

Write python code that finds the roots of the quadratic equation $ax^2 + bx + c$ where $a = 1$; $b = 5$; $c = 6$

```
In [87]: a = 1
b = 5
c = 6
x1 = (-b+(b**2-4*a*c)**0.5)/(2*a)
x2 = (-b-(b**2-4*a*c)**0.5)/(2*a)
myroots = {"x1":x1, "x2":x2}
myroots
```

```
Out[87]: {'x1': -2.0, 'x2': -3.0}
```

```
In [ ]:
```

```
In [ ]:
```

Functions

Write python function that finds the roots of a quadratic equation $ax^2 + bx + c$

```
In [2]: def root_quad(a,b,c):
D = b**2 -4*a*c
x1 = (-b+ D**0.5)/2*a
x2 = (-b- D**0.5)/2*a
myroot = {"x1":x1, "x2":x2}
return myroot
```

```
In [3]: root_quad(1,5,6)
```

```
Out[3]: {'x1': -2.0, 'x2': -3.0}
```



```
In [4]: root_quad(3,5,2)
```

```
Out[4]: {'x1': -6.0, 'x2': -9.0}
```

```
In [5]: root_quad(1,7,5)
```

```
Out[5]: {'x1': -0.8074175964327481, 'x2': -6.192582403567252}
```

```
In [6]: root_quad(3,9,6)
```

```
Out[6]: {'x1': -9.0, 'x2': -18.0}
```

```
In [ ]:
```

Write a function that takes an integer and check if it is even

```
In [7]: def even_int(n):  
        mod = n%2  
        return mod
```

```
In [8]: even_int(6)
```

```
Out[8]: 0
```

```
In [9]: even_int(22)
```

```
Out[9]: 0
```

```
In [11]: def root_quad(a,b,c):  
        import math  
        D = b**2 -4*a*c  
        x1 = (-b + math.sqrt(D))/2*a  
        #x1 = (-b+ D**0.5)/2*a  
        x2 = (-b- D**0.5)/2*a  
        myroot = {"x1":x1,"x2":x2}  
        return myroot
```

```
In [12]: root_quad(1,5,6)
```

```
Out[12]: {'x1': -2.0, 'x2': -3.0}
```

```
In [15]: math.sqrt(9)
```

```
Out[15]: 3.0
```

```
In [14]: import math
```

```
In [16]: import math as m
```

```
In [17]: m.sqrt(9)
```

```
Out[17]: 3.0
```

```
In [18]: m.sin(0.34)
```

```
Out[18]: 0.3334870921408144
```

```
In [19]: from math import sin, cos, tan, sqrt
```

```
In [20]: sin(sqrt(0.234))
```

```
Out[20]: 0.46508928447476977
```

```
In [ ]:
```

```
In [ ]:
```

In algebra you learnt about the n th roots of unity, defined as

$$z_k = \cos\left(\frac{2\pi k}{n}\right) + i \sin\left(\frac{2\pi k}{n}\right)$$

, $k = 0, 1, \dots, n - 1$ Write code to find the values of the 5th roots of unity.

```
In [33]: def nth_unity(n):  
    from math import cos, sin, pi  
    z0 = complex(cos(2*pi*0/n) , sin(2*pi*0/n))  
    z1 = complex(cos(2*pi*1/n) , sin(2*pi*1/n))  
    z2 = complex(cos(2*pi*2/n) , sin(2*pi*2/n))  
    z3 = complex(cos(2*pi*3/n) , sin(2*pi*3/n))  
    z4 = complex(cos(2*pi*4/n) , sin(2*pi*4/n))  
    return(z0,z1,z2,z3,z4)
```

```
In [32]: nth_unity(5)
```

```
Out[32]: (1+0j)
```

```
In [34]: nth_unity(5)
```

```
Out[34]: ((1+0j),  
          (0.30901699437494745+0.9510565162951535j),  
          (-0.8090169943749473+0.5877852522924732j),  
          (-0.8090169943749475-0.587785252292473j),  
          (0.30901699437494723-0.9510565162951536j))
```

```
In [38]: def nth_root_of_unity(n):  
          from math import sin, cos, pi  
          for k in range(n):  
              zi = complex(cos(2*pi*k/n), sin(2*pi*k/n))  
              print(zi)
```

```
In [44]: nth_root_of_unity(5)
```

```
(1+0j)  
(0.30901699437494745+0.9510565162951535j)  
(-0.8090169943749473+0.5877852522924732j)  
(-0.8090169943749475-0.587785252292473j)  
(0.30901699437494723-0.9510565162951536j)
```

```
In [43]: list(range(5))
```

```
Out[43]: [0, 1, 2, 3, 4]
```

```
In [1]: 21 + 45
```

```
Out[1]: 66
```

```
In [7]: for i in range(5):  
          x = i**2  
          print(x)
```

```
0  
1  
4  
9  
16
```

Indexing

```
In [45]: a = [1,2,3,4,5,6,7,8,9,1000]
```

```
In [49]: a[0]+a[5]+a[8]
```

```
Out[49]: 16
```

```
In [52]: b = list(range(1,1001))  
b[533]
```

Out[52]: 534

```
In [55]: len(b)
```

Out[55]: 999

```
In [58]: b[len(b)-2]
```

Out[58]: 999

```
In [59]: b[-1]
```

Out[59]: 1000

```
In [60]: b[-2]
```

Out[60]: 999

```
In [61]: b[-3]
```

Out[61]: 998

```
In [62]: a
```

Out[62]: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

```
In [63]: a[::-1]
```

Out[63]: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

```
In [64]: a_new = a
```

```
In [65]: a
```

Out[65]: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

```
In [66]: a_new
```

Out[66]: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

```
In [68]: a = [1,2,3,4,5,6,7,8,9]
b = a
b.append(10)
print(a)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
In [70]: a = [1,2,3,4,5,6,7,8,9]
b = a.copy()
b.append(10)
print(a)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
In [71]: a = [1,2,3,4,5,6,7,8,9]
a.append([10,11,12])
print(a)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, [10, 11, 12]]
```

```
In [72]: a = [1,2,3,4,5,6,7,8,9]
a.extend([10,11,12])
print(a)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

```
In [ ]:
```

```
In [ ]:
```

```
In [74]: a = [1,2,3,4,1,1,5,2,4]
```

```
In [75]: a.count(1)
```

```
Out[75]: 3
```

```
In [76]: a.index(1)
```

```
Out[76]: 0
```

```
In [81]: a.index(1,4)
```

```
Out[81]: 4
```

```
In [82]: a[50]
```

IndexError

Traceback (most recent call last)

Cell In[82], line 1

----> 1 a[50]

IndexError: list index out of range

```
In [100]: a = [1,2,3,4,5,6]
          b = ["a","b","c"]
          c = b.copy()
```

```
In [101]: d = a+b
          d
```

```
Out[101]: [1, 2, 3, 4, 5, 6, 'a', 'b', 'c']
```

```
In [102]: a.extend(b)
```

```
In [103]: a
```

```
Out[103]: [1, 2, 3, 4, 5, 6, 'a', 'b', 'c']
```

```
In [104]: print(a == d)
```

True

```
In [105]: e = 5*b
```

```
In [106]: e
```

```
Out[106]: ['a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c']
```

```
b.extend(e)
b
```

```
['a',
 'b',
 'c',
 'a',
 'b',
 'c',
 'a',
 'b',
 'c',
 'a',
 'b',
 'c',
 'a',
 'b',
 'c',
 'a',
 'b',
 'c']
```

```
b.extend(e)
b
```

[illegible]

```
b.extend(e)
b
```

[illegible]


```
In [110]: b.extend(e)  
b
```

[illegible]

```
'b',  
'c']
```

```
In [111]: print(b == e)
```

False

```
In [ ]:
```

```
In [ ]:
```

Strings

```
In [112]: a = 'This is a Python string'  
print(a.upper())
```

THIS IS A PYTHON STRING

```
In [113]: print(a.lower())
```

this is a python string

```
In [114]: print('P is located at index ' + str(a.index('P')))
```

P is located at index 10

```
In [115]: print(a.replace('Python', 'Cython'))
```

This is a Cython string

```
In [116]: print('endswith the text String? ' + str(a.endswith('String')))
```

endswith the text String? False

```
In [117]: print('endswith the text String? ' + str(a.endswith('string')))
```

endswith the text String? True